



# Data Handling: Import, Cleaning and Visualisation

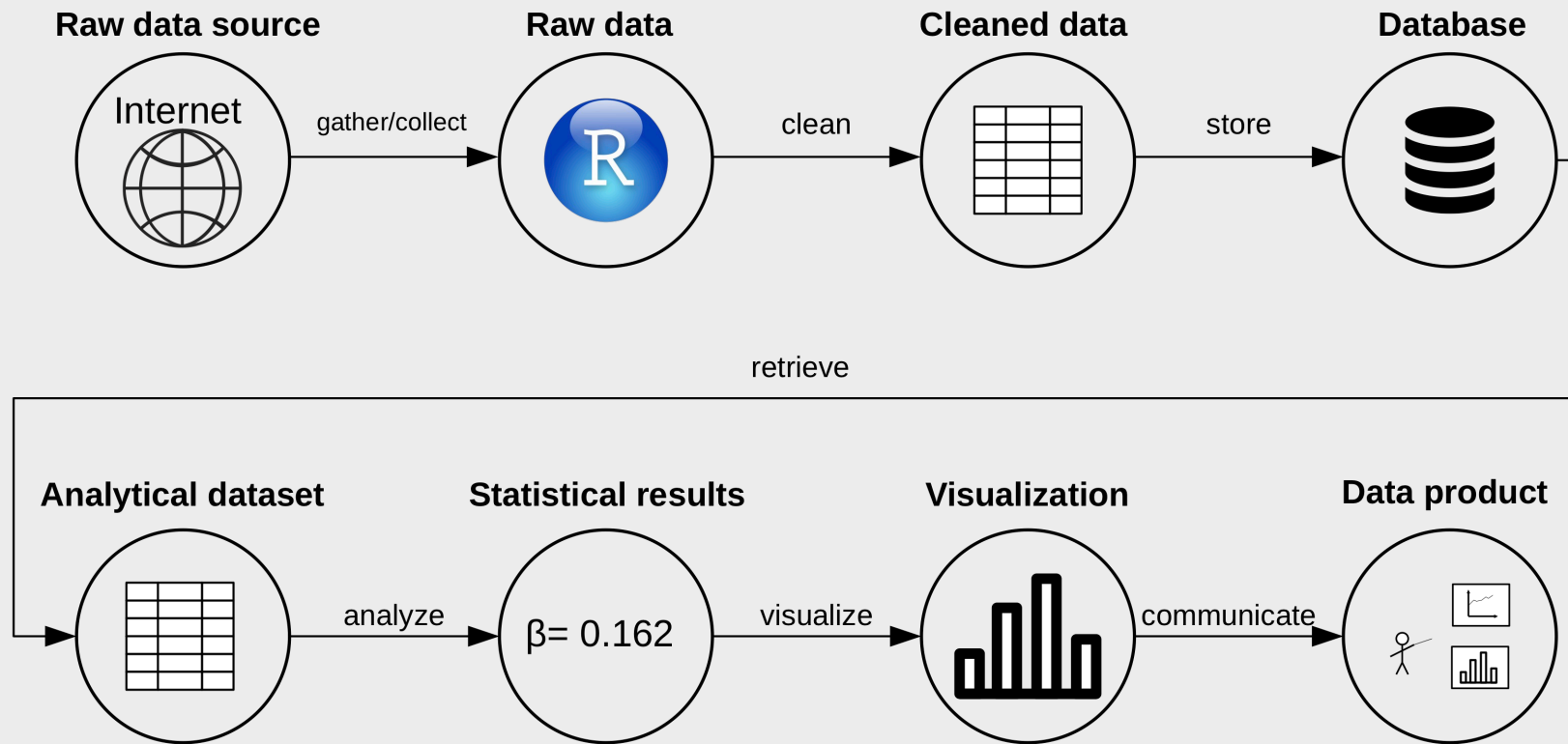
## Lecture 2: Programming with Data

Dr. Aurélien Sallin

2025-10-02

Recap

# Data (science) pipeline



At the end of the course, you will be able to...

- **Use essential tools for working with data**

We will work with the programming language R, but the concepts apply to any programming language.

- **Manage a data project from start to finish**

You will practice collecting, cleaning, and analyzing data to carry out a complete project in Economics (research, consulting, ...).

- **Formulate meaningful questions from data**

You will learn how to explore datasets and identify relevant questions.

- **Communicate insights effectively**

You will gain experience in presenting results clearly and persuasively.

# When you think of data science, you think of...



# Course organization

# Replacement session



To compensate for the session we missed on September 19th, please note the following changes to the schedule:

- Thursday, October 9th, 10:15–12:00: Lecture
- Thursday, October 9th, 16:15–18:00: Lecture (instead of exercises)
- **Friday, October 10th, 16:15–18:00, Room 01-011: Exercise Session 2 with Federica**

# Examination for HSG students in exchange

For HSG-students currently abroad on an exchange semester (not exchange students at HSG):

- Decentral exams as announced on the fact sheet are mandatory.
- I am allowed to offer an alternative examination as a sign of **goodwill** and **fairness (exception!)**.

## Oral exam as alternative

Exchange students will have the possibility to take an **oral exam** during the central examination period. Students in exchange must email me before **October 10th, 2025** to register.

## Summary of the examination process for exchange students:

- Central examination (70%): as planned, with all course participants.
- Group project (15%): completed remotely with your group.
- Decentral exam (15%): oral examination during the central examination period.



# Group formation for the group project

## ✗ Option 1: Random group formation

- Students will be randomly assigned to groups of five.

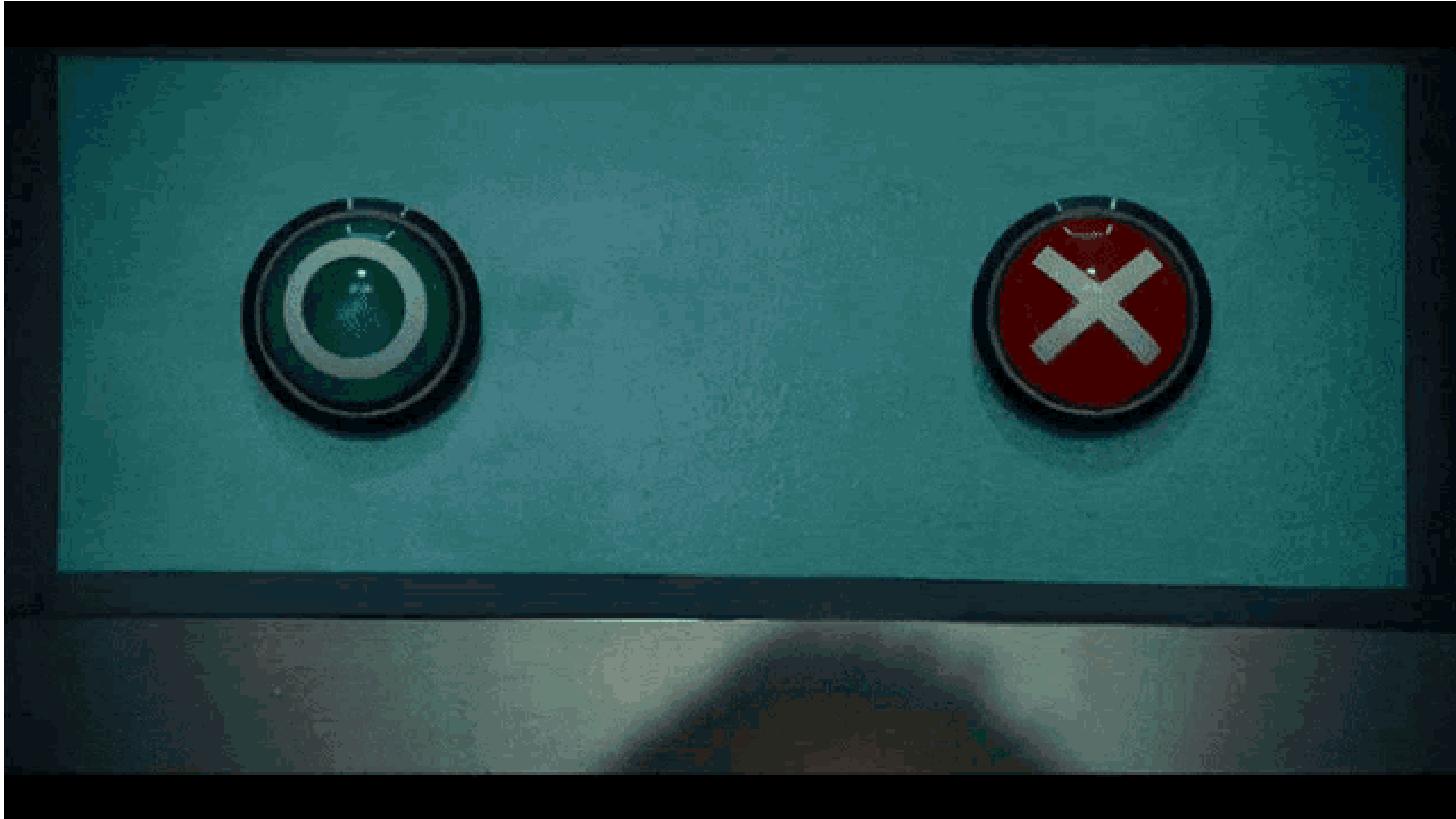
## ● Option 2: Students create their own groups

- Groups with fewer than five students will be completed by the instructors.
- Students who do not join a group will be randomly assigned.

### Important for both options:

- Depending on class size, some groups may have fewer or more than five members.
- Groups are **compulsory** for the group project submission.
- Once created, groups remain **fixed** for the entire project. No reshuffling is allowed.

Now, the vote:



Today

# Basic Programming Concepts in R

## Goals:

- Understand the basic programming concepts in R
- Write simple R scripts using loops, functions, and conditionals
- Understand the concept of functionals and how to use them

# The R environment

A collection of objects. These may be variables, arrays of numbers, character strings, functions, or more general structures built from such components.

# Values and Variables

Assign values to objects using `<-`.

# Vectors

R has two types of vectors:

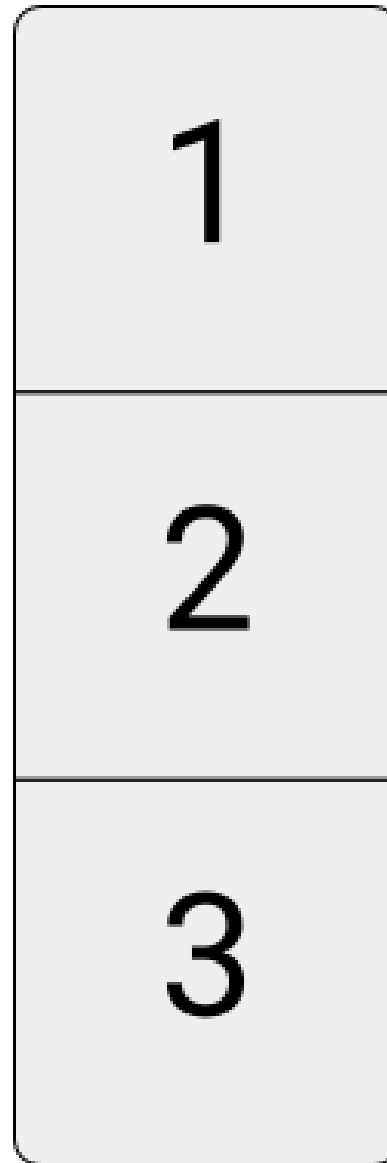
- **atomic vectors**: homogeneous collections of the same type 
- **lists**: heterogeneous collections of any type of R object, even other lists

# Vectors

The four types of atomic vectors that are used the most are: **logical, integer, double, character**.



# Vectors



# Vectors

|        |
|--------|
| Andy   |
| Brian  |
| Claire |

# Matrices

|   |   |   |
|---|---|---|
| 1 | 4 | 7 |
| 2 | 5 | 8 |
| 3 | 6 | 9 |

# Matrices are atomic vector arranged in 2D form

Homogeneous vectors with an added `dim` attribute that gives it rows and columns.

```
1 cbind(c(1,2,3), c(4,5,6), c(7,8,9))
2 rbind(c(1,4,7), c(2,5,8), c(3,6,9))
3 matrix(nrow = 3, ncol = 3, 1:9)
4
5 dim(cbind(c(1,2,3), c(4,5,6), c(7,8,9)))
```

|      | [,1] | [,2] | [,3] |
|------|------|------|------|
| [1,] | 1    | 4    | 7    |
| [2,] | 2    | 5    | 8    |
| [3,] | 3    | 6    | 9    |

# Matrices are atomic vector arranged in 2D form

Homogeneous vectors with an added `dim` attribute that gives it rows and columns.

```
1 cbind(c(1,2,3), c(4,5,6), c(7,8,9))
2 rbind(c(1,4,7), c(2,5,8), c(3,6,9))
3 matrix(nrow = 3, ncol = 3, 1:9)
4
5 dim(cbind(c(1,2,3), c(4,5,6), c(7,8,9)))
```

|      | [,1] | [,2] | [,3] |
|------|------|------|------|
| [1,] | 1    | 4    | 7    |
| [2,] | 2    | 5    | 8    |
| [3,] | 3    | 6    | 9    |

# Matrices are atomic vector arranged in 2D form

Homogeneous vectors with an added `dim` attribute that gives it rows and columns.

```
1 cbind(c(1,2,3), c(4,5,6), c(7,8,9))
2 rbind(c(1,4,7), c(2,5,8), c(3,6,9))
3 matrix(nrow = 3, ncol = 3, 1:9)
4
5 dim(cbind(c(1,2,3), c(4,5,6), c(7,8,9)))
```

|      | [,1] | [,2] | [,3] |
|------|------|------|------|
| [1,] | 1    | 4    | 7    |
| [2,] | 2    | 5    | 8    |
| [3,] | 3    | 6    | 9    |

# Matrices are atomic vector arranged in 2D form

Homogeneous vectors with an added `dim` attribute that gives it rows and columns.

```
1 cbind(c(1,2,3), c(4,5,6), c(7,8,9))
2 rbind(c(1,4,7), c(2,5,8), c(3,6,9))
3 matrix(nrow = 3, ncol = 3, 1:9)
4
5 dim(cbind(c(1,2,3), c(4,5,6), c(7,8,9)))
```

```
[1] 3 3
```

# Operators



# Most Operators are vectorized in R

```
vect <- c(1, 2, 3, 4)  
vect + 10
```

```
[1] 11 12 13 14
```

 When a function or operator is called vectorized, it means that it can take a whole vector as input and apply the operation element by element, without writing an explicit loop.

# Math operators: basic arithmetic

```
# Basic arithmetic
2+2
4*5
20/5
sum_result <- 2+2
sum_result -2
```

```
# Modulo (remainder)
5 %% 3

# Integral division
5 %/% 3
```

```
[1] 4
```

```
[1] 20
```

```
[1] 4
```

```
[1] 2
```

```
[1] 2
```

```
[1] 1
```

# Comparison operators

Comparison operators in R (Source: Shawn Santo)

| Operator               | Comparison               | Vectorized?                |
|------------------------|--------------------------|----------------------------|
| <code>x &lt; y</code>  | less than                | Yes                        |
| <code>x &gt; y</code>  | greater than             | Yes                        |
| <code>x &lt;= y</code> | less than or equal to    | Yes                        |
| <code>x &gt;= y</code> | greater than or equal to | Yes                        |
| <code>x != y</code>    | not equal to             | Yes                        |
| <code>x == y</code>    | equal to                 | Yes                        |
| <code>x %in% y</code>  | contains                 | Yes (over <code>x</code> ) |

# Logical/Booleans operators

Logical operators in R (Source: Shawn Santo)

| Operator                    | Operation                                            | Vectorized? |
|-----------------------------|------------------------------------------------------|-------------|
| <code>x   y</code>          | or                                                   | Yes         |
| <code>x &amp; y</code>      | and                                                  | Yes         |
| <code>!x</code>             | not                                                  | Yes         |
| <code>x    y</code>         | or*                                                  | No          |
| <code>x &amp;&amp; y</code> | and*                                                 | No          |
| <code>xor(x, y)</code>      | exclusive or <code>(x   y) &amp; !(x &amp; y)</code> | Yes         |

# Logical operators

Question:

Given  $x = 1$  and  $y = 6$ , what is:  $x < 5 \ \& \ y < 5$ ?

# Other common operators and functions

```
sqrt(4^2)  
log(2)  
exp(10)  
log(exp(10))
```

# Set operators

## Set operators in R

| Operator                    | Operation    | Vectorized? |
|-----------------------------|--------------|-------------|
| <code>union(x,y)</code>     | union        | Yes         |
| <code>intersect(x,y)</code> | intersection | Yes         |
| <code>setdiff(x, y)</code>  | difference   | Yes         |
| <code>setequal(x, y)</code> | equality     | No          |

# Sequence operators

The colon operator `:` generates regular sequences. It is equivalent to `seq(from, to)` in the `seq()` function.

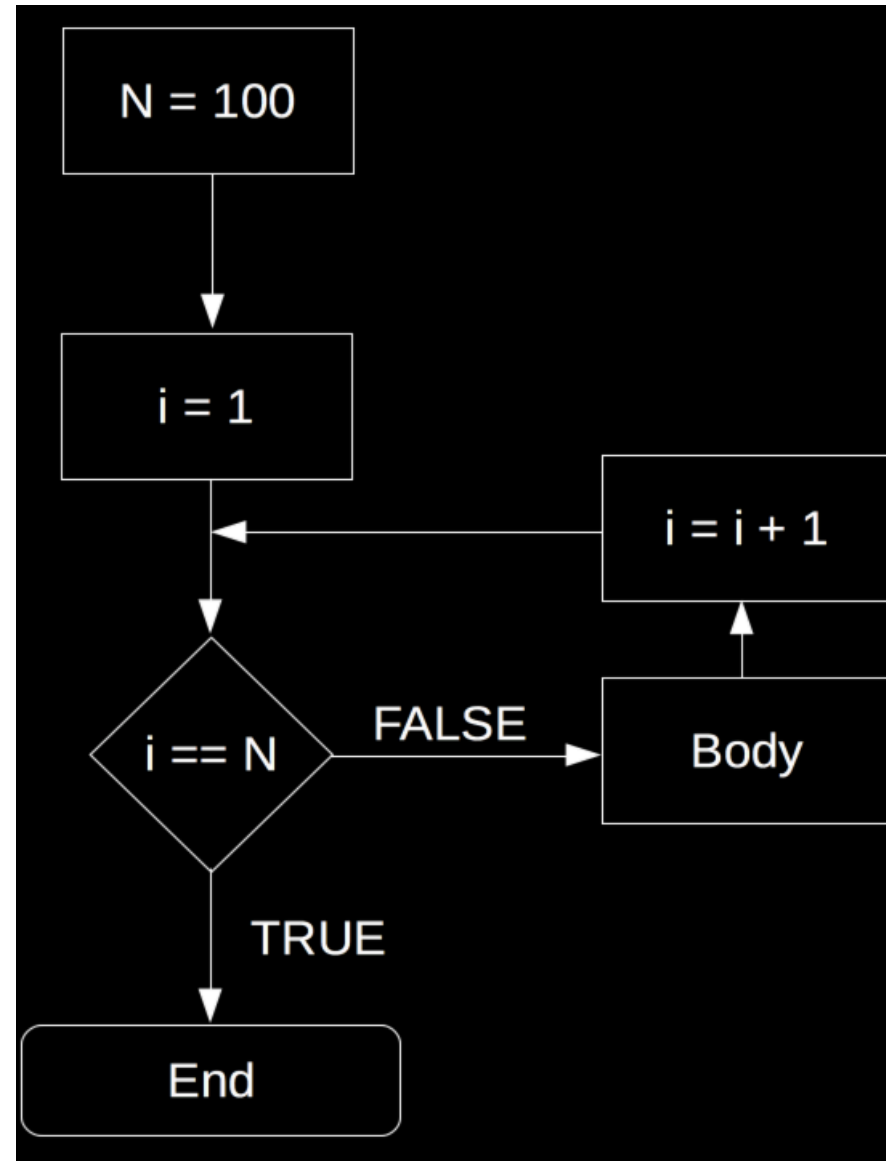
```
1:10
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```



# Loops

# for-loop



# for-loop

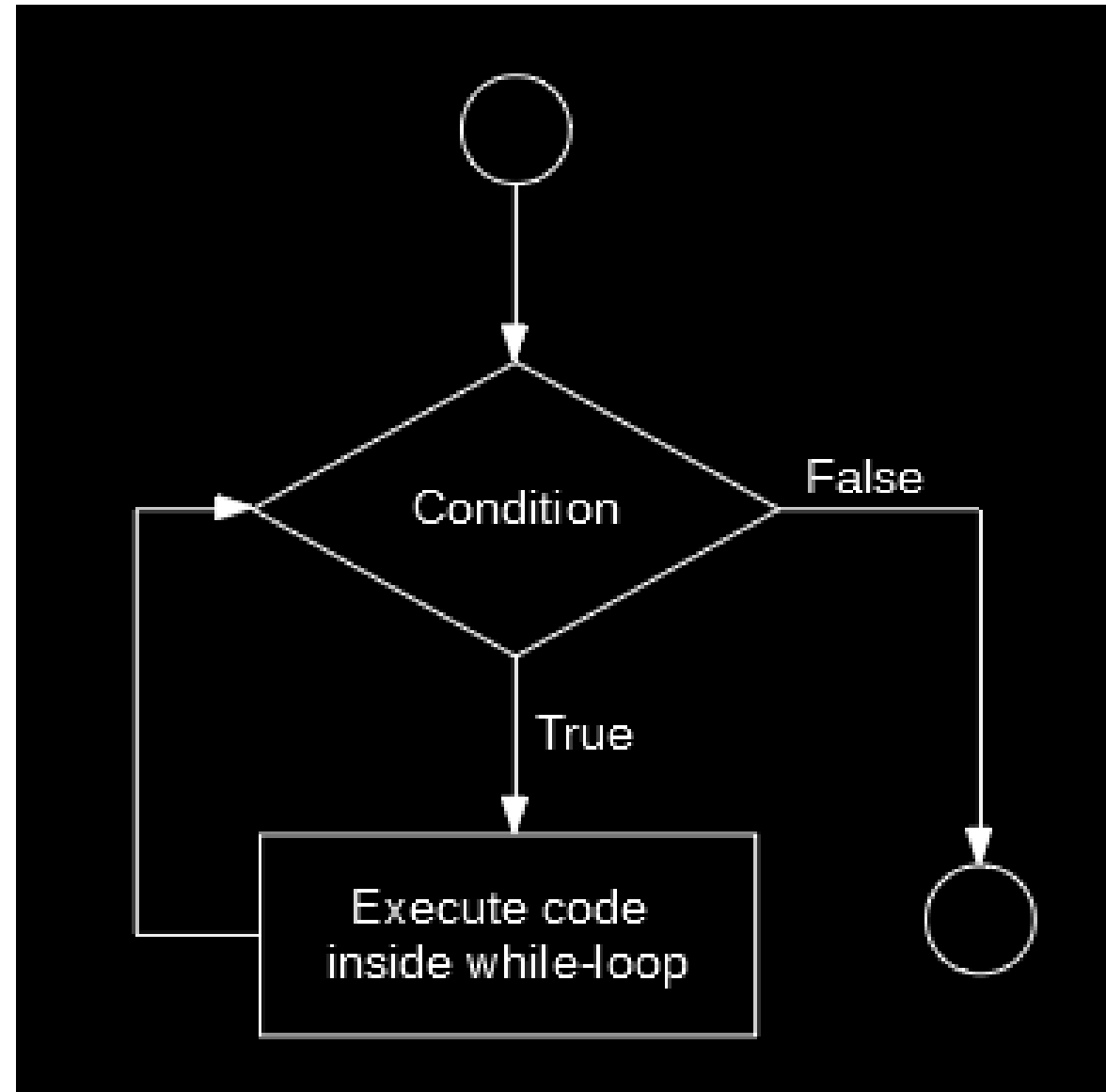
```
# number of iterations
n <- 100

# start loop
for (i in 1:n) {

  # BODY

}
```

# while-loop



# while-loop

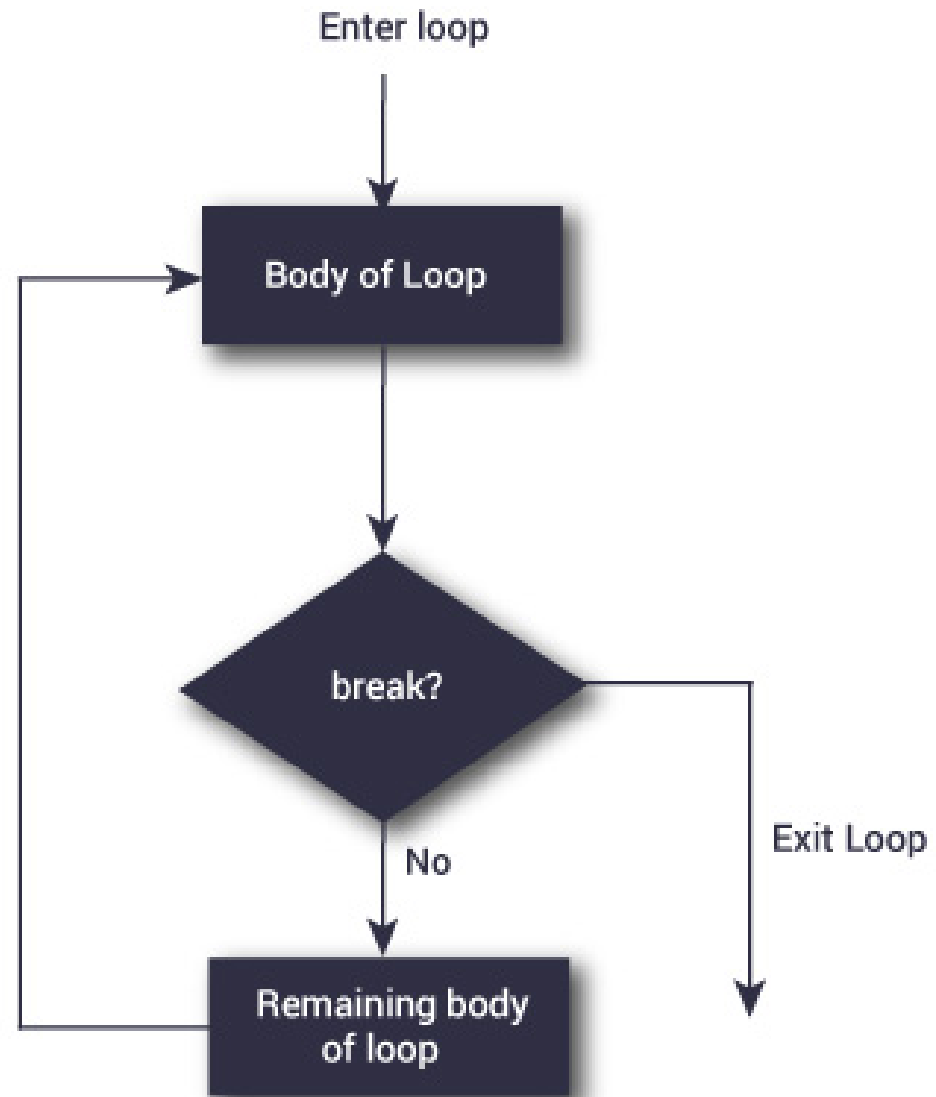
```
# initiate variable for logical statement
x <- 1

# start loop
while (x == 1) {

  # BODY

}
```

# repeat-loop



# repeat-loop

```
# Initiate variable
x <- 1

# Start repeat loop
repeat {

  x = x + 1

  # Break statement
  if (x == 6){
    break
  }
}
```

# Logical and control statements



# Control statements

```
condition <- TRUE

if (condition) {
  print("This is true!")
} else {
  print("This is false!")
}
```

```
[1] "This is true!"
```

# Functions

# Functions

$$f : X \rightarrow Y$$

# Functions

$$2 \times X = Y$$

# Functions in R

Functions in R are either **built-in** or **user-defined**.

Load built-in functions from a R-package:

```
# install a package
install.packages("<PACKAGE NAME>")

# load a package
library(<PACKAGE NAME>)
```

# Functions in R

Functions have three elements:

1. **formals()**, the list of arguments that control how you call the function
2. **body()**, the code inside the function
3. **environment()**, the data structure that determines how the function finds the values associated with the names (not the focus of this course)

# Functions in R

```
myfun <- function(x, y){  
  
  # BODY  
  z <- x + y  
  
  # What the function returns  
  return(z)  
}
```

## Formals

```
formals(myfun)
```

\$x

\$y

# Functions in R

```
myfun <- function(x, y){  
  
  # BODY  
  z <- x + y  
  
  # What the function returns  
  return(z)  
}
```

## Body

```
body(myfun)
```

```
{  
  z <- x + y  
  return(z)  
}
```



# Functions in R

```
myfun <- function(x, y){  
  
  # BODY  
  z <- x + y  
  
  # What the function returns  
  return(z)  
}
```

## Environment

```
environment(myfun)
```

```
<environment: R_GlobalEnv>
```

# Functions in R

A simple power function

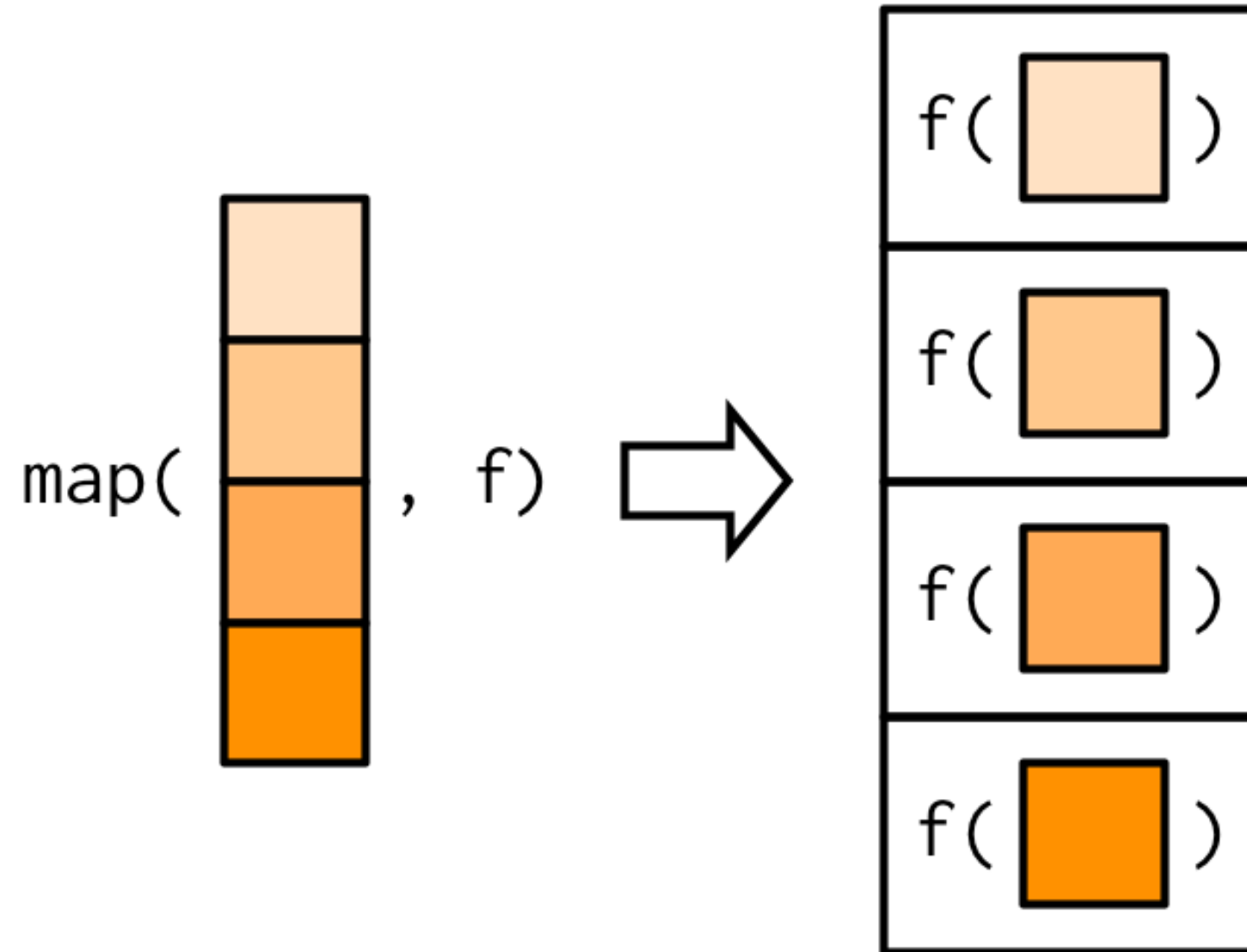
```
powerFunction <- function(base, exponent){  
  results <- base ^ exponent  
  return(results)  
}  
  
powerFunction(exponent = 2, base = 3)  
powerFunction(base = 2, exponent = 3)  
powerFunction(2, 3)  
powerFunction(c(2,4,3), 3)
```

# Step-up your game: Functionals

# Functionals

- A functional is a function that takes a function as an input and returns a vector or a list as output.
- Functionals are alternative to for-loops.
- Functionals can be programmed using the `apply` family (`apply`, `lapply`, `tapply`), or the `purrr::map()` family.

# Functionals: representation with `map()`



# Example of a functional

```
# Install purrr and load
# library(purrr)

# Set a user-defined function
triple <- function(x) x * 3

# With lapply
lapply(1:3, triple)
```

```
[[1]]
[1] 3

[[2]]
[1] 6

[[3]]
[1] 9
```

```
# Apply the function to a vector. map_dbl Returns a vector
map_dbl(1:3, triple)
```

```
[1] 3 6 9
```

# The “apply” functional

`apply` applies a function to columns or rows of a matrix. It loops over rows (`MARGIN = 1`) or columns (`MARGIN = 2`) of a matrix.

```
# Empty matrix with 2 rows and 4 columns
mymatrix <- matrix(c(1,2,3,11,12,13,1,10),
                  nrow = 2,
                  ncol = 4)
```

```
# Apply the sum function on each column
apply(mymatrix, MARGIN = 2, sum)
```

```
# Apply the sum function on each row
apply(mymatrix, MARGIN = 1, sum)
```

```
      [,1] [,2] [,3] [,4]
[1,]     1     3    12     1
[2,]     2    11    13    10
```

```
[1]  3 14 25 11
```

```
[1] 17 36
```

# Recap



# Loops and functionals

| for-loops       | functionals                                                                                |
|-----------------|--------------------------------------------------------------------------------------------|
| Gets messy fast | Elegant and readable, allow for better syntax                                              |
| Long code       | Short code, based on functions                                                             |
| Standard base R | <code>apply</code> family: standard R. <code>map</code> family: <code>purrr</code> package |

# Tutorials

# Tutorial 1: A Function to Compute the Mean

Write a function `meanN(x)` that computes the mean of a numeric vector. We are not allowed to call it “mean” because `mean()` is already a function in R.

Starting point: we should be aware of how the mean is defined:

$$\bar{x} = \frac{1}{n} \left( \sum_{i=1}^n x_i \right) = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

# Tutorial 1: A Function to Compute the Mean

```
#####  
# Mean Function:  
# Computes the mean, given a  
# numeric vector.  
  
meanN <- function(x){  
  
}
```

# Tutorial 2: on slow and fast sloths

We can use loops to simulate natural processes over time. Write a program that calculates the populations of two kinds of sloths over time. At the beginning of year 1, there are **1000 slow sloths** and **1 fast sloth**. This one fast sloth is a new mutation that is genetically able to use roller blades. Not surprisingly, being fast gives it an advantage, as it can better escape from predators.



A slow sloth



A fast sloth in its natural element

# Tutorial 2: on slow and fast sloths

Each year, each sloth has one offspring. There are no further mutations, so slow sloths beget slow sloths, and fast sloths beget fast sloths. Also, each year 40% of all slow sloths die each year, while only 30% of the fast sloths do.

At the beginning of year one there are 1000 slow sloths. Another 1000 slow sloths are born. But, 40% of these 2000 slow sloths die, leaving a total of 1200 at the end of year one. Meanwhile, in the same year, we begin with 1 fast sloth, 1 more is born, and 30% of these die, leaving 1.4.

| Beginning of Year | Slow Sloths | Fast Sloths |
|-------------------|-------------|-------------|
| 1                 | 1000        | 1           |
| 2                 | 1200        | 1.4         |
| 3                 | 1440        | 1.96        |

Enter the first year in which the fast sloths outnumber the slow sloths.

# Tutorial 3: A loop function

Be the function

```
appendsums <- function(lst){  
  
  #Repeatedly append the sum of the current last three elements  
  #of lst to lst.  
  
}
```

Create a function that repeatedly appends the sum of the current last three elements of the vector `lst` to `lst`. Hint: use the `append` and `tail` functions. Your function should loop 25 times. To check if your function is correct, run:

```
sum_three = c(0, 1, 2)  
appendsums(sum_three)  
  
# Solution for testing:  
sum_three[10] == 125
```

# Questions?